

Reviewer Pack — Diophantine Solution Count Bounds Resolution Length in 3-SAT

Entry 00e75e5287ed · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 14:32:11 UTC

Diophantine Solution Count Bounds Resolution Length in 3-SAT

Entry ID: 00e75e5287ed **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 14:32:11 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Approximation **Field B** (complexity object): Resolution Proof Length

Statement:

For a 3-SAT instance with n variables, the minimal resolution proof length is at most the number of integer solutions to the system S derived from the instance, where S consists of equations enforcing clause truth via linear Diophantine constraints modulo a prime $p \geq n+1$.

Rationale (proposer's reasoning):

Diophantine equations capture combinatorial constraints, and their solution counts may reflect the complexity of resolving logical contradictions. Modular arithmetic introduces structure that could correlate with proof complexity via bounded arithmetic frameworks.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 57aa2a237d0b8c1d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from math import gcd

def extended_gcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        g, x, y = extended_gcd(b % a, a)
        return (g, y - (b // a) * x, x)

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError('Modular inverse does not exist')
    else:
        return x % m

def chinese_remainder_theorem(n, a):
    sum = 0
    prod = 1
    for ni in n:
        prod *= ni
    for ni, ai in zip(n, a):
        p = prod // ni
        sum += ai * mod_inverse(p, ni) * p
    return sum % prod

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda k: abs(A[k][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        factor = A[i][i]
        for j in range(n):
            A[i][j] /= factor
        b[i] /= factor
        for k in range(m):
            if k != i:
```

```

        factor = A[k][i]
        for j in range(n):
            A[k][j] -= factor * A[i][j]
            b[k] -= factor * b[i]
    return [b[i] for i in range(m)]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    p = n + 1
    clauses = [[random.choice([-i, i]) for _ in range(3)] for _ in range(n)]

    # Convert clauses to Diophantine equations modulo p
    A = []
    b = []
    for clause in clauses:
        eq = [0] * (2*n + 1)
        for var in clause:
            if var > 0:
                eq[var - 1] += 1
            else:
                eq[-var - 1] -= 1
        A.append(eq[:-1])
        b.append(-eq[-1])

    # Solve the system using Gaussian elimination
    solutions = gaussian_elimination(A, b)

    # Count integer solutions modulo p
    solution_count = 0
    for i in range(p):
        if all((sol + i) % p == 0 for sol in solutions):
            solution_count += 1

    # Generate a random 3-SAT instance and find its resolution proof length
    variables = list(range(1, n+1))
    assignment = {var: random.choice([True, False]) for var in variables}
    def evaluate_clause(clause):
        return any((assignment[var] if var > 0 else not assignment[-var]) for var in clause)

    proof_length = 0
    stack = []
    while True:
        unsatisfied_clauses = [i+1 for i, clause in enumerate(clauses) if not evaluate_clause(clause)]
        if not unsatisfied_clauses:
            break
        unit_clause = next((c for c in clauses if len([var for var in c if abs(var) in assignment])), None)
        if unit_clause is None:
            proof_length += 1
            continue
        var = [v for v in unit_clause if v in assignment][0]
        stack.append((var, not assignment[var]))
        assignment[var] = not assignment[var]

    # Check if the resolution proof length is less than or equal to the number of solutions
    conjecture_holds = proof_length <= solution_count

```

```

return {
    "metric_name": "resolution_proof_length",
    "metric_value": proof_length,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Proof length {proof_length} > solution co
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]),
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d675fda.py", line 125, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6d675fda.py", line 64, in run_trial
    clauses = [[random.choice([-i, i]) for _ in range(3)] for _ in range(n)]
               ^

```

UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unbound local variable 'i', preventing data collection | next: Fix the code error in line 64 where 'i' is used without definition

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	82007
2	preregistration	ollama_remote	qwen3:8b	0	0	24176
3	novelty	ollama_remote	qwen3:8b	0	0	20655
4	novelty	ollama_remote	qwen3:8b	0	0	15190
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35027
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14224
7	judge	ollama_remote	qwen3:8b	0	0	15699

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 206978 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/00e75e5287ed.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/00e75e5287ed.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/00e75e5287ed.tar.gz (if generated)